1. Proposito del Documento

Este documento describe la arquitectura de seguridad de alto nivel del ecosistema de gestión de formularios. Detalla los componentes principales (capa estática, gateway de lógica de negocio, APIs, ORM y sistema de logging), las decisiones de diseño fundamentales y los flujos de información que garantizan un funcionamiento seguro y auditable.

2. Visión General del Sistema

El sistema es una aplicación web para la gestión de formularios de compra con un fuerte enfoque en la seguridad y la auditoría. Los principales componentes son:

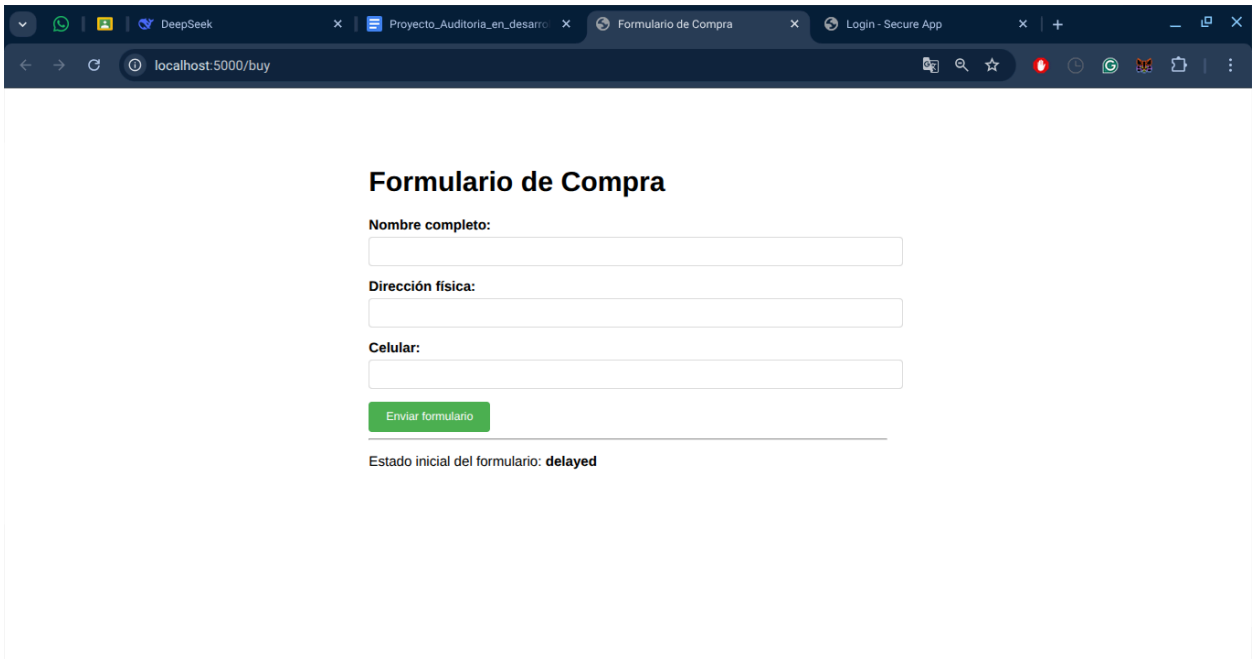
- Capa Estática (Frontend): Proporciona las interfaces de usuario.
- Vistas Principales: /buy (Formulario de compra), /who (Información).
- Vistas de Configuración: Para la gestión de parámetros del sistema (a definir).
- Vistas de Auditoría: Dashboards de seguridad en /security/dashboard y pruebas en /octomatrix/test.
- Gateway de Aplicación (Backend - Flask): Contiene toda la lógica de negocio y seguridad. Sirve las APIs y orquesta las interacciones entre componentes.
- APIs (Interfaz Programática): Endpoints RESTful que exponen la funcionalidad del sistema a la capa estática y a potenciales integraciones futuras.
- Capa de Persistencia (ORM - SQLAlchemy): Abstrae la interacción con la base de datos. Actualmente usa SQLite para persistencia (formularios) y estructuras en memoria para datos volátiles (usuarios, intentos de login).
- Sistema de Logging y Auditoría: Captura eventos de seguridad, traza actividades y alimenta los dashboards de auditoría.

Descripción de alto nivel del ecosistema: capa estática, gateway, APIs, ORM y logging.

- Capa estatica
  La capa estática consta de 3 vistas principales , vistas de configuración y vistas de auditoría y la vista de formularios

Vistas Principales
/buy

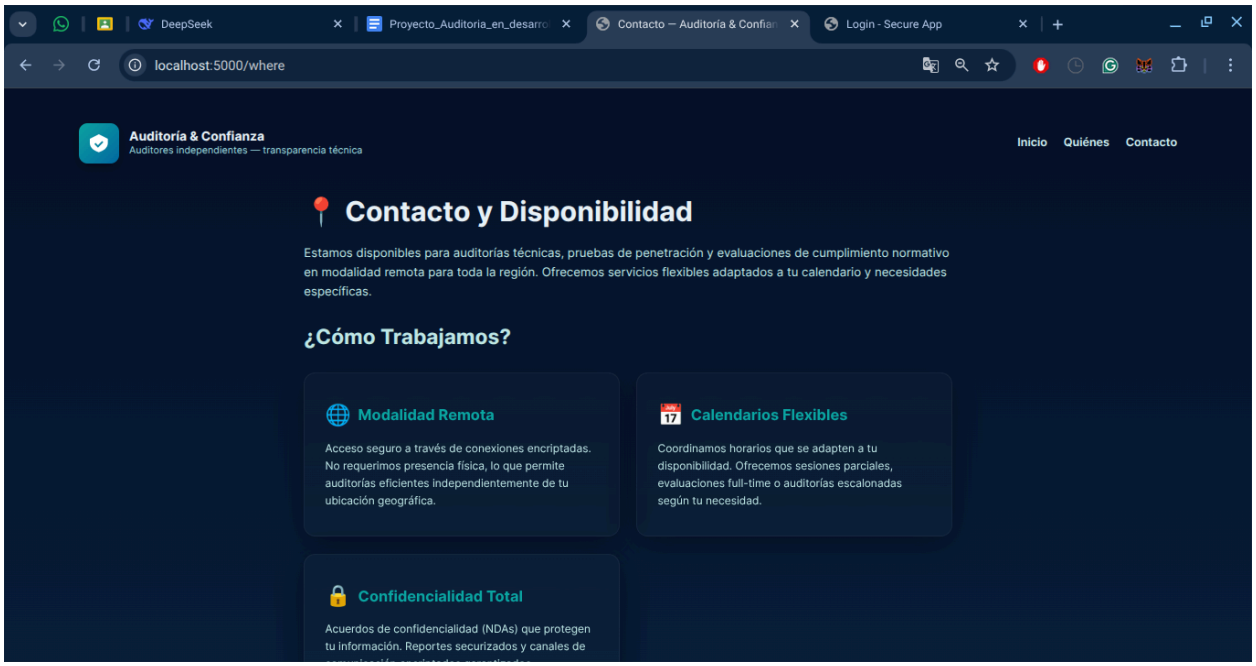
docs/arq/architecture_document.md



/who

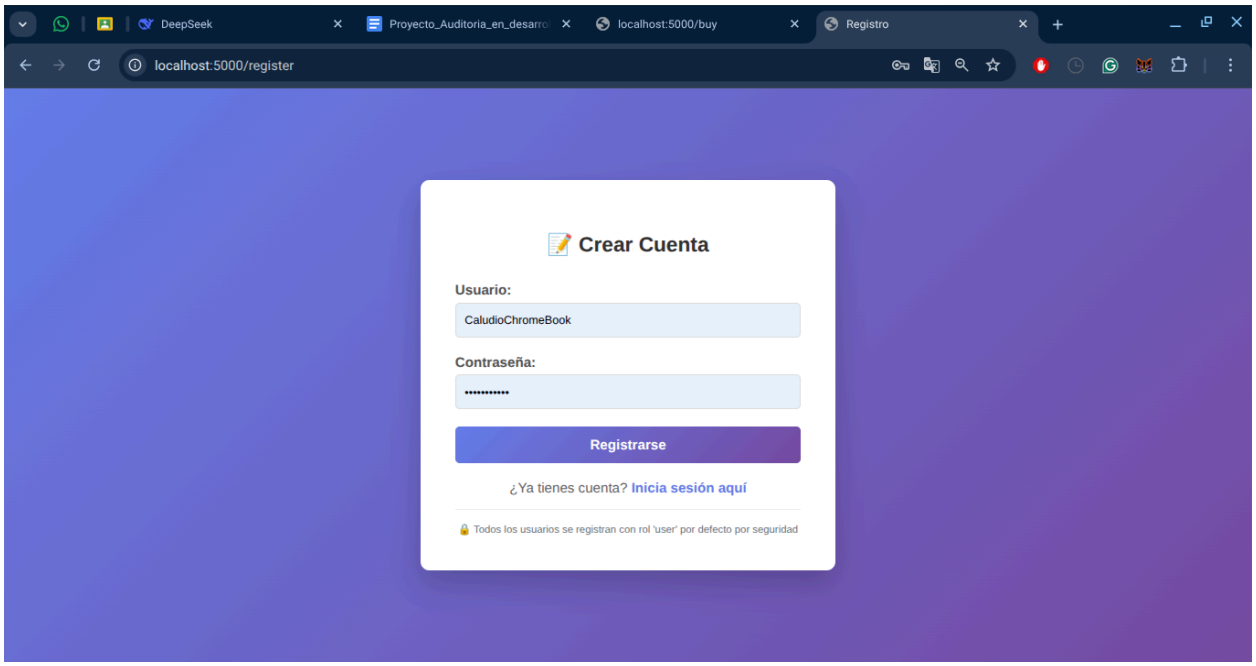
/where

docs/arq/architecture_document.md



VIstas de configuracion

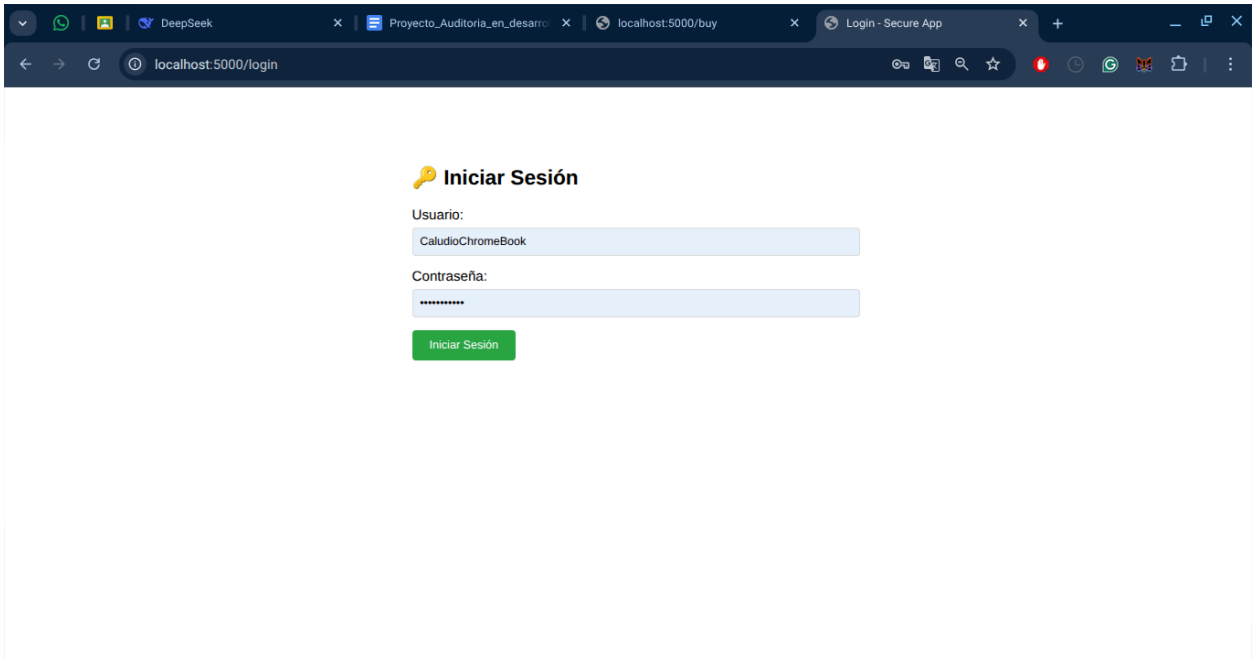
/register



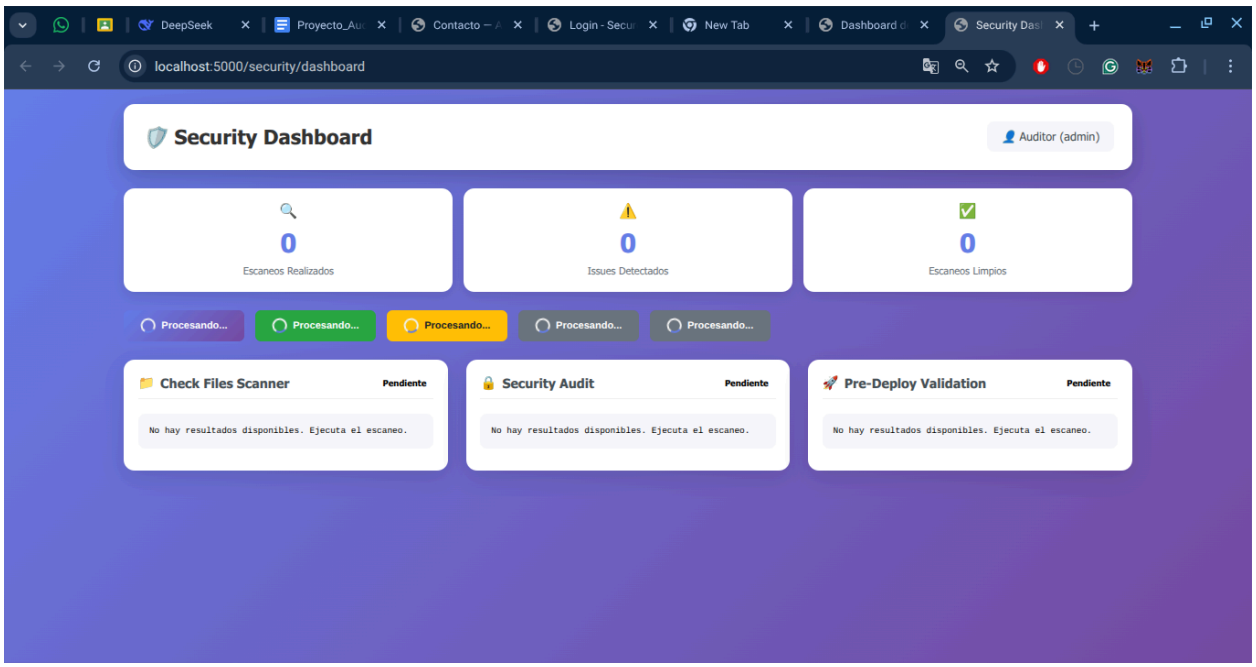
/login

docs/arq/architecture_document.md

VIsa

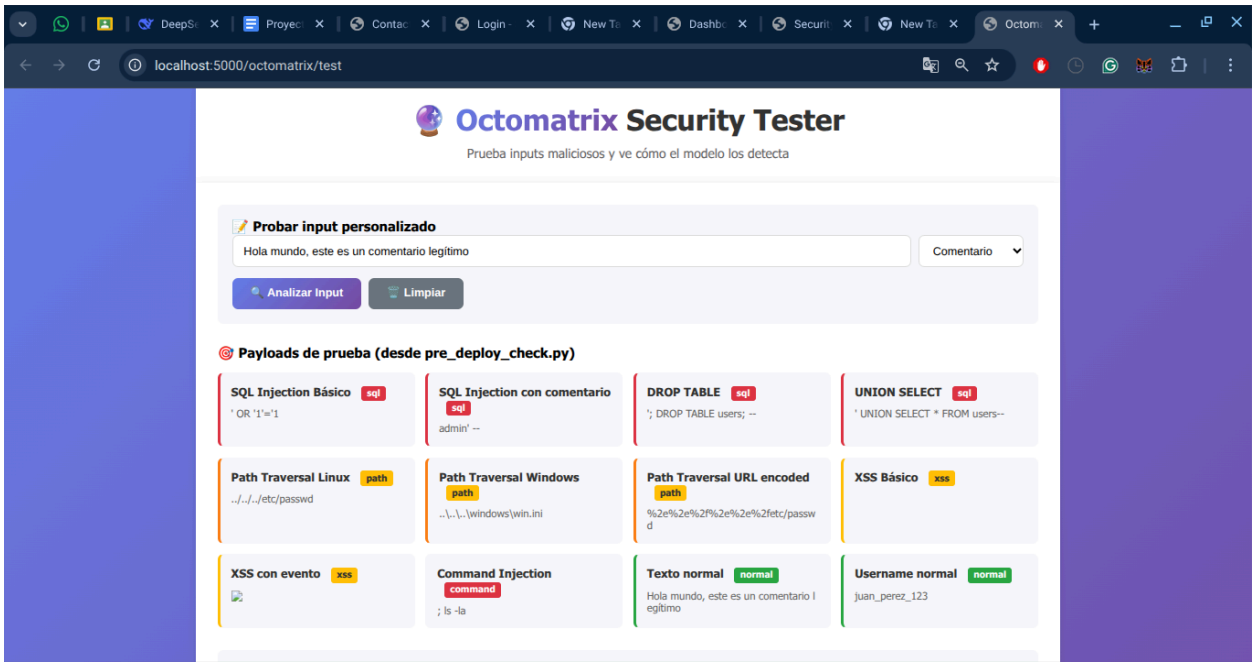


Vistas de auditoria
http://localhost:5000/security/dashboard

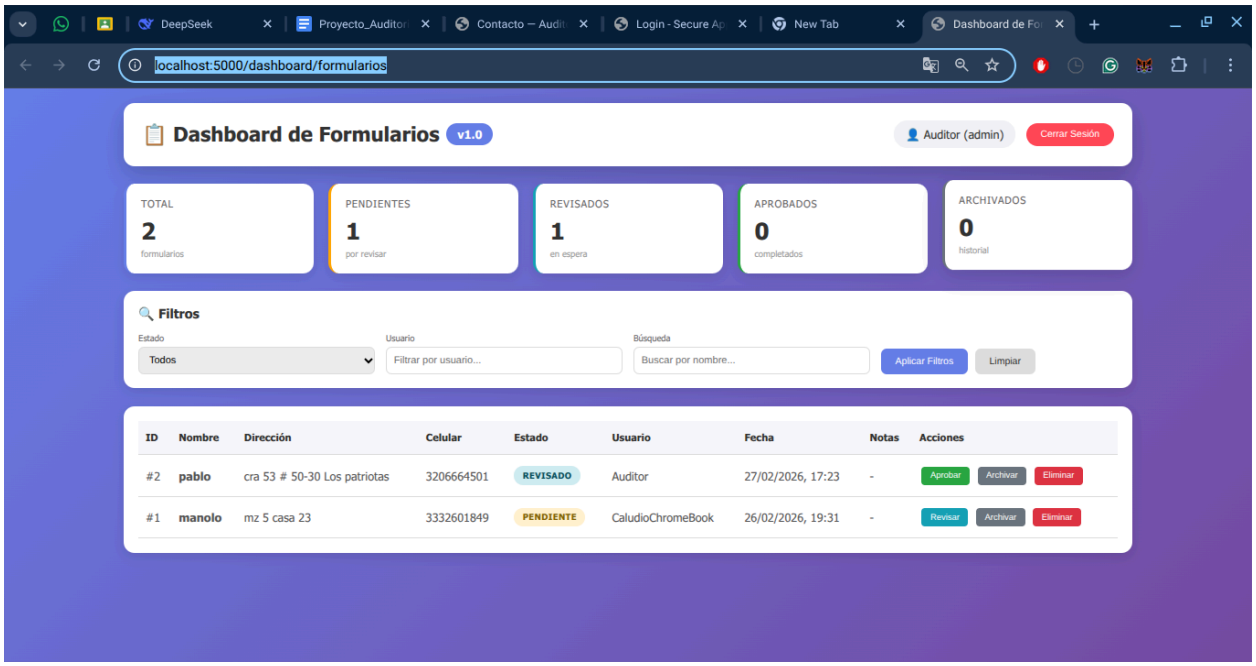


http://localhost:5000/octomatrix/test

docs/arq/architecture_document.md



Vistas de formulario
http://localhost:5000/dashboard/formularios



2. Vision especifica del sistema

Identificación de usuarios, operadores, administradores como auditores.

Modulo de autentificacion
El módulo de autenticación consta de un token basado en pbkdf2_hmac , este es generado desde el login y tiene una duración definida por la sesión , en futuras implementaciones se quiere usar la base de datos para guardar estos y que se actualizen cada 30 dias

Rol Operaciones, este tipo de usuarios interactuar con el orm aprobando o archivando los formularios en la vista de formularios

Administradores , este tipo de usuarios son solamente generados por el backend con una llave secreta denotada como ADMIN_CREATION_SECRET, estos pueden ejecutar los test de seguridad del sistema

3. Stakeholders y roles

Identificación de usuarios, operadores, administradores y auditores
El sistema define cuatro roles con privilegios claramente diferenciados, siguiendo el principio de mínimo privilegio.

Usuario (Anónimo/Autenticado): Interactúa con el sistema para crear y ver formularios.

Operador: Usuario con permisos para interactuar con el ORM a través de la vista de formularios, pudiendo aprobar o archivar los formularios existentes. Gestiona el flujo de negocio.

Administrador: Rol de máxima confianza. Son creados exclusivamente desde el backend mediante una llave secreta (ADMIN_CREATION_SECRET). Tienen la capacidad de ejecutar los test de seguridad del sistema y realizar acciones críticas en el flujo de formularios.

Auditor: Rol con acceso de solo lectura a los logs y al dashboard de seguridad (/security/dashboard) para supervisar la salud del sistema y detectar anomalías.

4. Principios de Diseño

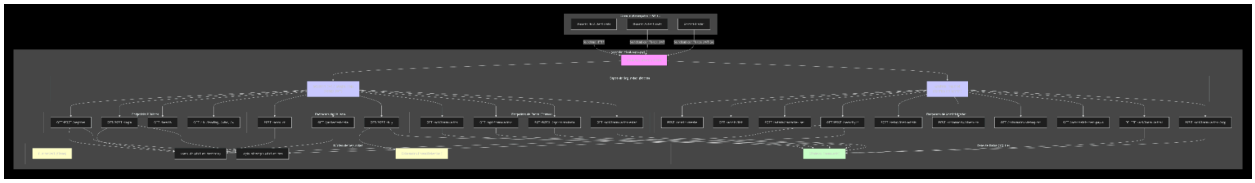
La arquitectura se ha construido sobre los siguientes pilares:

Separación de Responsabilidades: Cada componente (autenticación, lógica de negocio, persistencia, logging) tiene una función específica y bien definida.

Mínimo Privilegio: Los roles de usuario tienen los permisos estrictamente necesarios para cumplir su función.

Seguridad por Diseño: Las consideraciones de seguridad (como el sistema anti-timing attacks o el Octomatrix) están integradas en el núcleo de la aplicación, no son un añadido posterior.

Desacoplamiento de Infraestructura: El uso de ORM (SQLAlchemy) y variables de entorno permite migrar la aplicación a diferentes infraestructuras (Render, DigitalOcean, on-premise) con cambios mínimos.



Diseño final de arquitectura

5. Vista logica

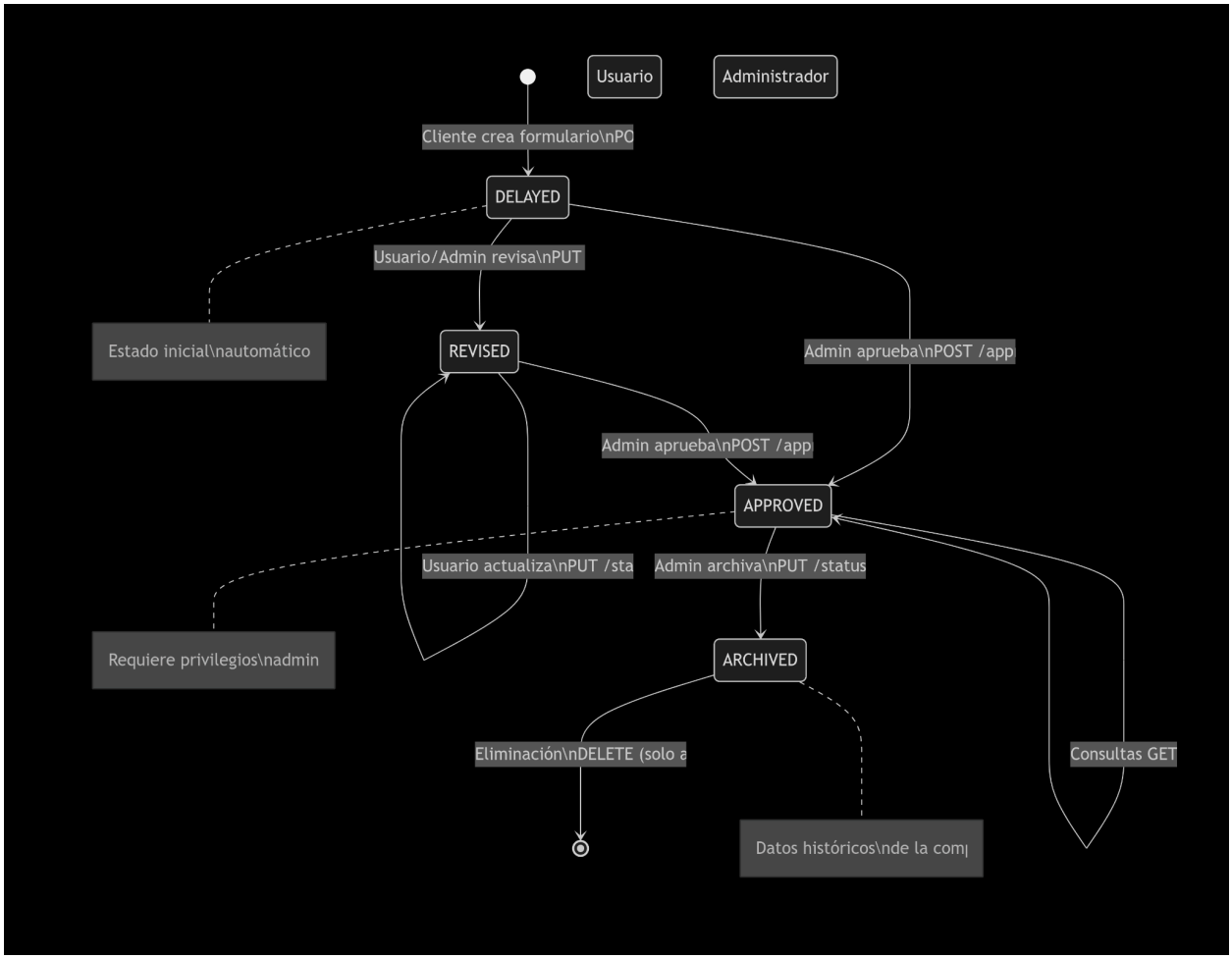
Flujo de Negocio (Máquina de Estados): Los formularios siguen un ciclo de vida estricto definido por un Enum. Esto garantiza la integridad de los datos y facilita la auditoría. Los estados son: DELAYED (inicial) → REVISED (revisado por operador) → APPROVED (aprobado por admin) → ARCHIVED (archivado).

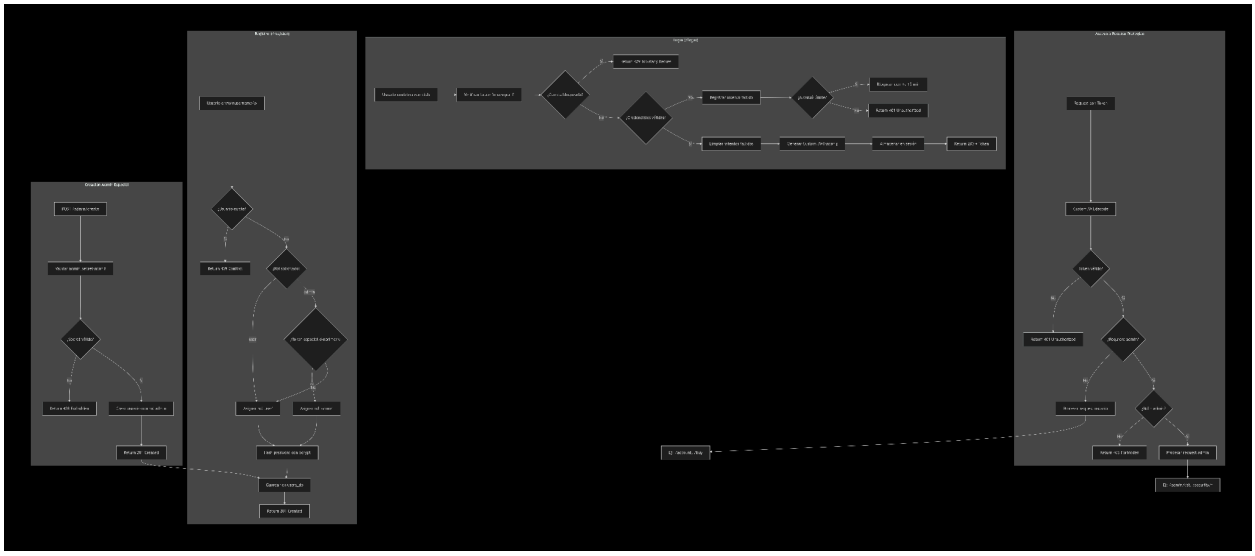
Flujo de Autenticación: Implementa múltiples capas de defensa:

Validación de credenciales contra users_db (pronto en base de datos) usando bcrypt.
Protección contra fuerza bruta mediante un contador de intentos por IP/usuario.
Generación de un JWT personalizado firmado con PBKDF2-HMAC-SHA256 (100,000 iteraciones) que incluye la IP del cliente en el payload para validación adicional.
Sesión de Flask persistente para mantener la sesión del lado del servidor.
Flujo de Octomatrix (Detección de Amenazas): Este módulo actúa como un WAF a nivel de aplicación. Analiza los inputs en tiempo real contra patrones de ataque conocidos (OWASP Top 10) y un modelo de Machine Learning (.pkl opcional). Si el "puntaje de amenaza" supera un umbral, la petición es bloqueada automáticamente (respuesta 403) y se registra el evento.

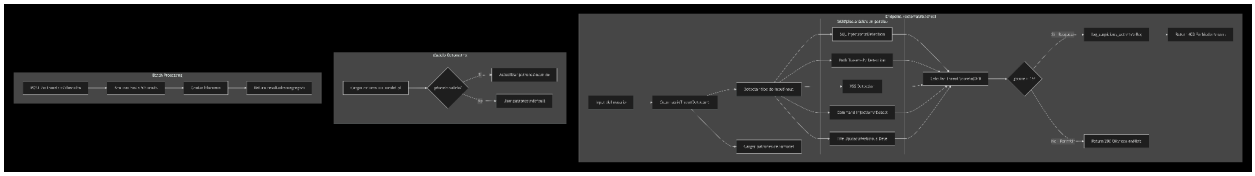
Flujo de Auditoría: Un sistema comprehensivo que captura eventos de todos los módulos (login, cambios de estado, bloqueos de Octomatrix), los correlaciona y aplica

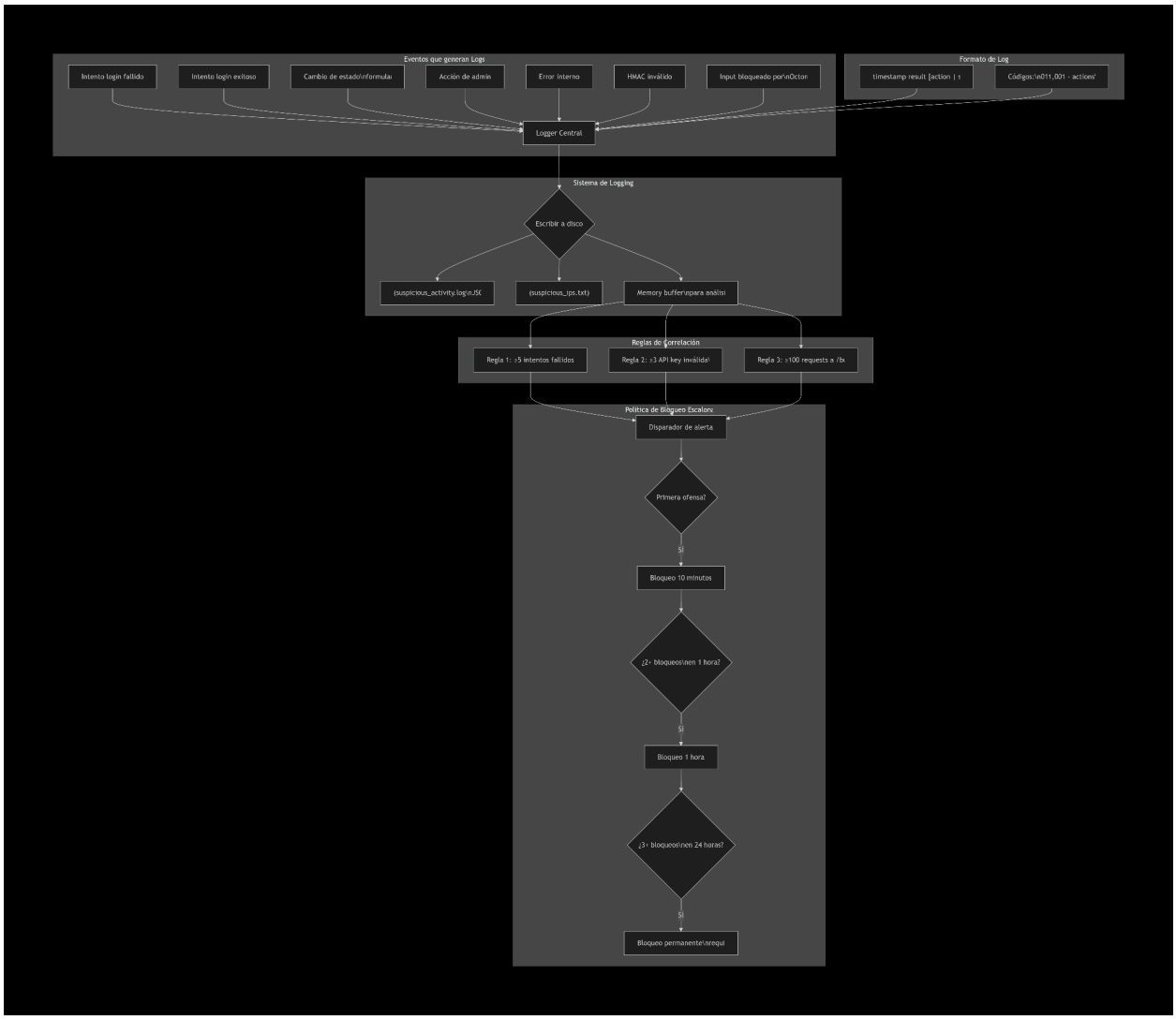
políticas de bloqueo escalonadas, alimentando el dashboard de seguridad en tiempo real.

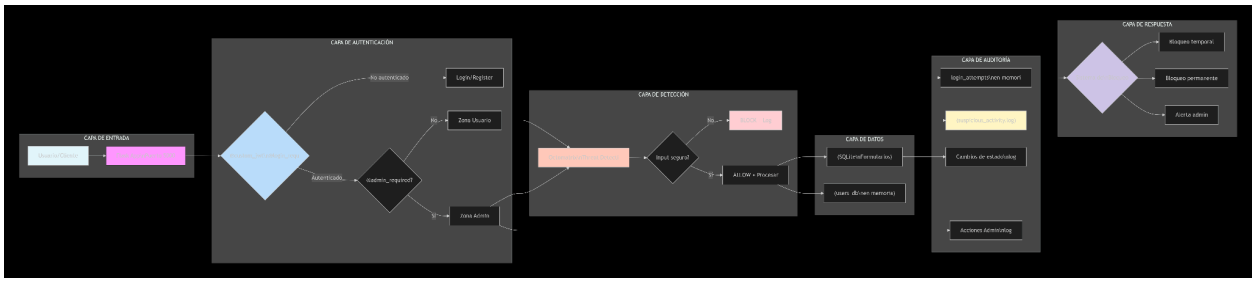




docs/arq/architecture_document.md







Resumen de los Flujos:

Flujo de Negocio: Los formularios pasan por estados claros (DELAYED → REVISED → APPROVED → ARCHIVED), con reglas específicas para cada transición y privilegios de admin para acciones críticas.

Flujo de Autenticación: Implementa múltiples capas:

Validación de credenciales con bcrypt

Protección contra brute force

JWT personalizado con PBKDF2

Roles de usuario estrictos

Flujo de Octomatrix: Analiza inputs en tiempo real contra múltiples tipos de amenazas, calcula un puntaje y bloquea automáticamente si supera el umbral.

Flujo de Auditoría: Sistema completo que captura eventos, los correlaciona y aplica políticas de bloqueo escalonadas basadas en el comportamiento.

Integración 360°: Todos los componentes trabajan juntos para crear un sistema seguro y auditable desde la entrada hasta el almacenamiento

6. Vista fisica / Deploy

Cómo se despliega en Render, DigitalOcean o entorno empresarial.
La aplicación está diseñada para ser desplegada como un contenedor único en plataformas como Render o DigitalOcean App Platform. La configuración se realiza mediante variables de entorno. Para entornos empresariales, se puede desplegar detrás de un proxy reverso (Nginx) con múltiples réplicas de la aplicación para alta disponibilidad, compartiendo una base de datos PostgreSQL y una caché Redis.

7. Seguridad de la Arquitectura

Controles de Acceso: Basados en roles (RBAC) implementados en los decoradores de las rutas de Flask.

Autenticación: Sistema híbrido (JWT + Sesión) con protección anti-fuerza bruta y creación segura de admins mediante HMAC.

Logging: Centralizado y estructurado, listo para ser ingerido por un SIEM.

Segregación de Redes: En producción, se recomienda que la base de datos y Redis no sean accesibles desde internet, solo desde la red interna de la aplicación.
Controles de acceso, autenticación, logging, segregación de redes.

Decisiones Arquitectónicas (ADR)

1. **Sistema de Autenticación: Custom JWT vs Librerías Estándar**

Decisión: Implementar un JWT personalizado con PBKDF2-HMAC-SHA256 en lugar de usar librerías estándar como `PyJWT` o `Flask-JWT`.

Contexto: Necesitábamos un sistema de tokens que fuera seguro pero que también nos permitiera tener control total sobre la implementación para auditorías de seguridad y cumplimiento OWASP Top 10.

Consecuencias:

- ✅ Control total sobre el algoritmo de firma (usamos PBKDF2 con 100,000 iteraciones en lugar del HMAC-SHA256 simple de JWT estándar)
- ✅ Eliminación de dependencias externas en la capa crítica de seguridad
- ✅ Posibilidad de incluir campos personalizados como `ip` en el payload para validación adicional

- 🔄 Mayor complejidad de mantenimiento, pero compensada por la transparencia total

- ⚠️ Responsabilidad de implementar correctamente la validación de expiración y estructura

2. **Protección de Endpoints Administrativos: HMAC para Creación de Admins**

Decisión: Usar `hmac.compare_digest()` para validar el `admin_secret` en lugar de una comparación de strings simple o un sistema de tokens complejo.

Contexto: La creación de administradores es una operación crítica que debe estar protegida contra timing attacks y fuerza bruta.

Consecuencias:

- ✅ Protección contra timing attacks mediante comparación de strings en tiempo constante

- ✅ Simplicidad: un solo secret en el entorno para operaciones administrativas

- ✅ Integración natural con variables de entorno (12-factor app)

- ⚠️ El secret debe ser rotado periódicamente y almacenado de forma segura

```python

# Ejemplo de la decisión en código

if not hmac.compare_digest(provided_secret, admin_secret):

    return jsonify({"error": "Invalid admin secret"}), 403

```

3. **Estrategia de Almacenamiento: Híbrida (SQLite + Memoria)**

Decisión: Combinar SQLite para datos persistentes (formularios) con estructuras en memoria (`users_db`, `login_attempts`) para datos volátiles de autenticación.

Contexto: Los datos de usuarios y sesiones cambian constantemente y requieren acceso rápido, mientras que los formularios necesitan persistencia y consultas estructuradas.

Consecuencias:

- ✅ Rendimiento óptimo para autenticación (sin latencia de disco)
- ✅ Reset automático de intentos de login al reiniciar (característica de seguridad)
- ✅ Datos de formularios persistentes y consultables vía ORM
- ⚠️ En entorno multi-instancia, la memoria compartida requeriría Redis
- 🔄 Fácil migración futura a PostgreSQL para formularios y Redis para sesiones

4. **Sistema de Detección de Amenazas: Octomatrix Integrado**

Decisión: Implementar detección de amenazas en tiempo real como capa de seguridad antes del procesamiento de negocio, usando un modelo híbrido (patrones estáticos + modelo ML opcional).

Contexto: Necesitábamos proteger endpoints críticos como `/buy` contra inyecciones y ataques sin depender exclusivamente de un WAF externo.

Consecuencias:

- ✅ Protección activa en la capa de aplicación
- ✅ Bloqueo inmediato de inputs maliciosos (respuesta 403)

- ✅ Capacidad de cargar modelos `.pkl` para mejorar detección
- ✅ Logging automático de actividades sospechosas para análisis forense
- ⚠️ Overhead de procesamiento en cada request (optimizable con caché)

5. **Máquina de Estados para Formularios**

Decisión: Implementar un `Enum` de estados (`FormStatus`) con transiciones controladas en lugar de un campo de texto libre.

Contexto: Los formularios de compra necesitan un ciclo de vida claro y auditable, con diferentes permisos según el estado.

Consecuencias:

- ✅ Integridad de datos: solo estados válidos
- ✅ Auditoría implícita: cada cambio de estado puede ser rastreado
- ✅ Lógica de negocio clara: transiciones definidas en el modelo
- ✅ Fácil generación de reportes y métricas por estado

```python
class FormStatus(enum.Enum):
    DELAYED = "delayed"      # Estado inicial
    REVISED = "revised"      # Revisado por usuario/operador
    APPROVED = "approved"    # Aprobado por admin
    ARCHIVED = "archived"    # Archivado (histórico)
```

Riesgos Conocidos

Riesgos Aceptados Conscientemente

1. **Pérdida de Datos de Usuarios en Memoria**

   - **Riesgo:** `users_db` y `login_attempts` se almacenan en memoria volátil. Un reinicio del servidor elimina todos los usuarios registrados y los intentos de login.

   - **Aceptación:** Aceptado para entorno de desarrollo/demo. En producción, se migrará a una base de datos persistente.

   - **Mitigación:** Endpoint `/setup/first-admin` y scripts de backup regulares.

2. **Single Point of Failure en la Aplicación**

   - **Riesgo:** Toda la lógica de autenticación, negocio y seguridad reside en una sola instancia de Flask.

   - **Aceptación:** Aceptado para MVP y cargas bajas. La aplicación es stateless en cuanto a sesiones (JWT), permitiendo escalado horizontal futuro.

   - **Mitigación:** Uso de JWT permite distribuir carga; el estado compartido requeriría Redis.

3. **Modelo Octomatrix No Entrenado**

   - **Riesgo:** Si `octomatrix_model.pkl` no existe o está corrupto, el sistema usa patrones predeterminados que pueden ser menos efectivos.

   - **Aceptación:** Aceptado como fallback seguro. Los patrones default cubren OWASP Top 10 básico.

- **Mitigación:** Logs de advertencia al iniciar, endpoint `/octomatrix/debug-model` para diagnóstico.

4. **Rate Limiting Básico por IP**

   - **Riesgo:** El sistema de bloqueo de login usa IP + usuario, pero un atacante distribuido (botnet) podría evadirlo.

   - **Aceptación:** Aceptado para protección básica. En producción, se complementará con Cloudflare o similar.

   - **Mitigación:** Política escalonada de bloqueos y logging de IPs sospechosas.

5. **SQLite en Producción**

   - **Riesgo:** SQLite no maneja bien alta concurrencia de escritura.

   - **Aceptación:** Aceptado para cargas bajas (demostraciones, desarrollo).

   - **Mitigación:** ORM SQLAlchemy permite migrar a PostgreSQL/MySQL cambiando solo la URI.

6. **JWT sin Refresh Tokens**

   - **Riesgo:** Los tokens duran 24 horas sin posibilidad de renovación automática.

   - **Aceptación:** Decisión deliberada para simplicidad. La sesión se mantiene vía cookie de Flask.

   - **Mitigación:** La sesión de Flask (`session.permanent = True`) maneja la persistencia del lado del servidor.

Evolución Futura

Mejoras Planeadas sin Comprometer la Arquitectura Actual

1. **Persistencia de Usuarios en Base de Datos**

- Migrar `users_db` de memoria a SQLite/PostgreSQL
- Mantener `login_attempts` en memoria o Redis para rendimiento
- Agregar tabla `users` con campos: id, username, password_hash, role, created_at, last_login, locked_until

2. **Sistema de Refresh Tokens**

- Implementar JWT de corta duración (15 min) + refresh token en HTTP-only cookie
- Mantener compatibilidad con tokens actuales durante transición
- Endpoint `/refresh` para renovar access token

3. **Octomatrix con Machine Learning Avanzado**

- Entrenar modelo con datasets públicos de ataques (OWASP, CVE)
- Implementar actualización en caliente del modelo sin reiniciar app
- Agregar detección de patrones ofuscados y polimórficos

4. **API Versionada y Documentación OpenAPI**

- Versión explícita en URLs: `/api/v1/formularios`
- Generar documentación interactiva con Swagger/OpenAPI
- Mantener retrocompatibilidad por al menos 2 versiones

5. **Sistema de Colas para Procesamiento Asíncrono**

- Usar Celery + Redis para tareas pesadas (escaneos de seguridad, reports)

- Endpoints inmediatos devuelven "accepted" con ID de tarea
- Endpoint `/tasks/<id>/status` para consultar progreso

6. **Webhooks para Eventos Críticos**

- Notificaciones cuando:
  - Admin bloqueado permanentemente
  - Formulario aprobado
  - Detección de ataque en curso
- Configurable por endpoint vía base de datos

7. **Dashboard de Seguridad en Tiempo Real**

- WebSockets para métricas en vivo
- Gráficos de intentos de ataque por IP/tipo
- Heatmap de actividades sospechosas

8. **Integración con SIEM Externo**

- Enviar logs estructurados a Splunk/ELK
- Formato estándar (CEF, LEEF) para correlación
- Alertas configurables en SIEM

9. **Multi-factor Authentication (MFA)**

- Soporte para TOTP (Google Authenticator)
- Endpoint `/mfa/setup` y `/mfa/verify`
- Mantener compatibilidad con autenticación simple actual

10. **Modo "Solo Lectura" para Mantenimiento**

- Endpoint `/admin/maintenance-mode`
- Permite consultas GET pero bloquea POST/PUT/DELETE
- Mensaje claro para clientes

11. **Exportación de Datos y Reportes**

- Generar PDF/CSV de formularios por rango de fechas
- Reportes programados vía cron jobs
- Endpoint `/api/reports/generate`

12. **Cache Inteligente con TTL por Rol**

- Cachear respuestas para endpoints públicos
- Invalidar caché en cambios de estado
- TTL más corto para admins, más largo para usuarios anónimos

Principios de Evolución

1. **Backward Compatibility:** Todas las mejoras mantendrán compatibilidad con la API actual
2. **Configuración sobre código:** Nuevas features serán configurables vía variables de entorno
3. **Migración gradual:** Implementación por fases, comenzando por features no disruptivas
4. **Observabilidad:** Cada nueva feature incluirá métricas y logs desde el día 1

docs/arq/architecture_document.md